

Red-Black Tree

Written Report

Student Name: Sam Kiselev

Student ID: 25231857

GitHub link: <https://github.com/SemenKis/Red-Black-Tree-Implementation>

Video Link: <https://drive.google.com/file/d/1hiWZGLO7B4Km-nVY9oFtEYGVNcvyHEki/view?usp=sharing>

C++ implementation and empirical study

(a) What I Built

For this project, I implemented a Red-Black Tree in C++. A Red-Black Tree is a self-balancing binary search tree where every node has a red or black colour. The colours and balancing rules prevent the tree from becoming heavily unbalanced.

My implementation stores integer values. Each node contains a key, its colour, and pointers to its parent, left child and right child. Missing children are represented using nullptr instead of separate NIL sentinel nodes.

The project separates the Red-Black Tree class definition and implementation into RedBlackTree.hpp and RedBlackTree.cpp, with main.cpp used for examples. The main operations are insertion, deletion, search, in-order traversal, left and right rotations, and balancing after insertion and deletion.

Insertion first follows normal Binary Search Tree rules. The balance() method then restores the Red-Black Tree properties using recolouring and rotations. Deletion first performs the structural BST deletion and calls fixRulesAfterRemoval() when removing a black node may have violated the Red-Black properties.

I kept the final Red-Black Tree implementation integer-based rather than using templates. This kept the focus on the balancing algorithm and made the implementation easier to navigate.

(b) Track A - Empirical Study

For the empirical study, I compared my Red-Black Tree implementation with an AVL Tree. I chose an AVL Tree because both are self-balancing binary search trees, but they use different approaches to maintaining balance.

My hypothesis was that the Red-Black Tree would perform better for insertion, while the AVL Tree would perform better for searching. AVL Trees maintain stricter balance, which can reduce tree

height and potentially improve searching. Red-Black Trees allow more variation in tree height, which can reduce the amount of restructuring required during insertion.

I tested both implementations using the same randomly ordered integer values. I measured insertion and search times for datasets containing 1,000, 5,000, 10,000, 50,000 and 100,000 elements. Execution time was measured in microseconds. I repeated the experiment multiple times and used the average results to reduce the effect of timing variation.

Elements	RB Insert (μ s)	AVL Insert (μ s)	RB Search (μ s)	AVL Search (μ s)
1,000	144	160	31	27
5,000	815	840	178	177
10,000	1,427	1,425	297	284
50,000	7,630	9,562	2,780	2,274
100,000	16,409	21,119	6,879	5,783

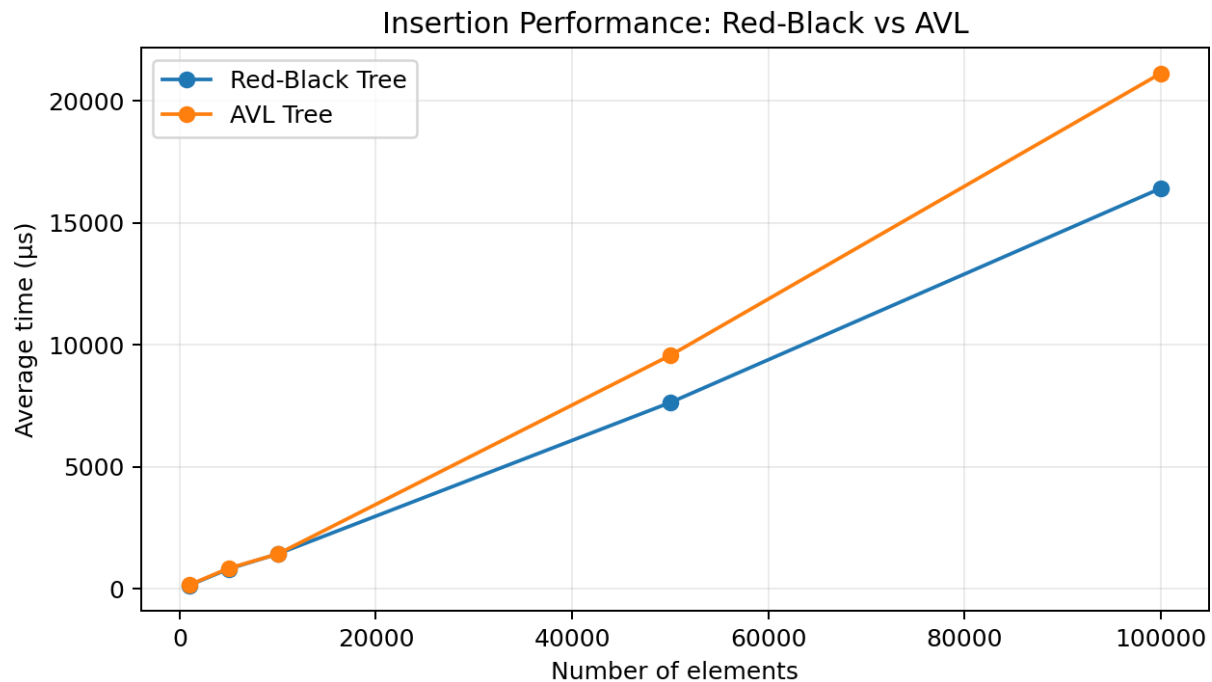


Figure 1. Average insertion time for Red-Black and AVL Trees.

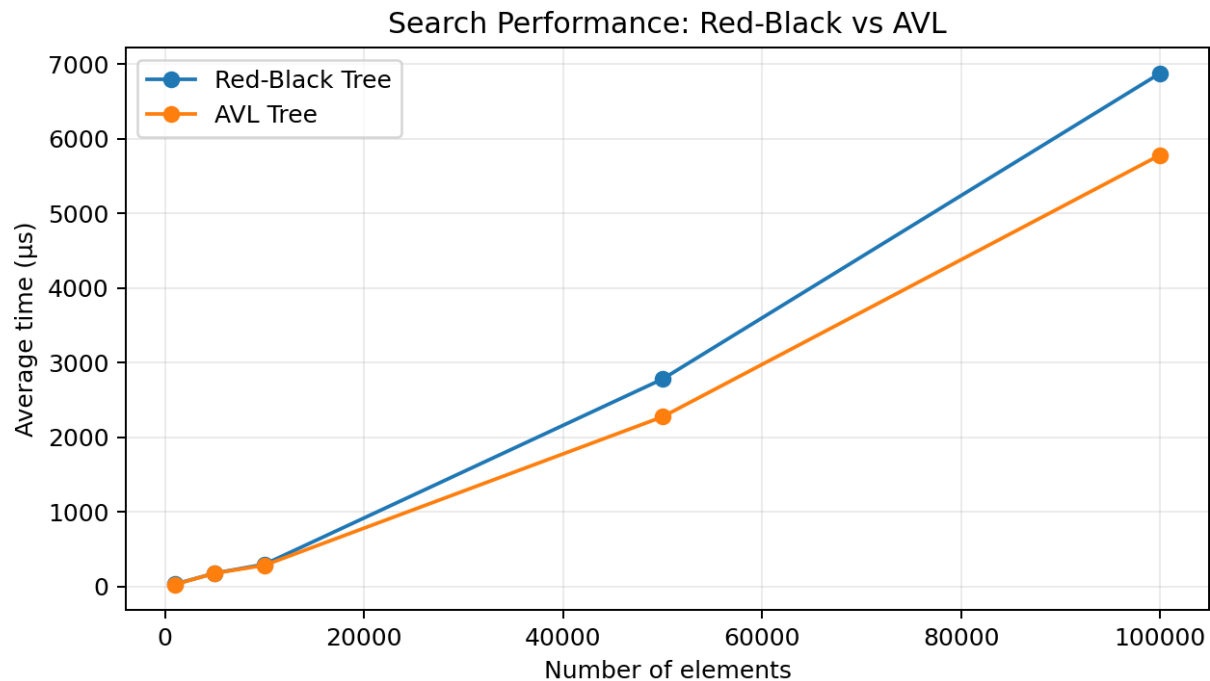


Figure 2. Average search time for Red-Black and AVL Trees.

The results showed little difference for the smaller datasets, but a clearer pattern appeared as the number of elements increased. At 100,000 elements, the Red-Black Tree required approximately 16,409 μs for insertion compared with 21,119 μs for the AVL Tree. For searching, the Red-Black Tree required approximately 6,879 μs , while the AVL Tree required approximately 5,783 μs .

These results support my hypothesis for the larger workloads tested. My Red-Black Tree implementation performed better for insertion, while the AVL implementation performed better for searching. However, this does not mean that one tree is always faster. At 10,000 elements, for example, insertion performance was almost identical. The results depend on the implementation, input data and workload.

The results can be explained by the different balancing strategies. AVL Trees maintain stricter balance, which can reduce search depth, while Red-Black Trees allow a less strictly balanced structure and can require less restructuring during updates. The experiment therefore demonstrated why neither structure completely replaces the other.

(c) What I Learned

One of the most important things I learned was that a Red-Black Tree cannot simply be considered a replacement for an AVL Tree. Before this project, I did not properly understand the practical trade-off between the two structures. The benchmark made this clearer: for the larger datasets I tested, my Red-Black Tree was faster for insertion, while the AVL Tree was faster for searching.

This was particularly interesting because I implemented my own AVL Tree a few years ago. Working on a Red-Black Tree this time gave me an opportunity to compare two approaches to self-balancing trees rather than learning them separately.

I also learned that ideas from different balancing approaches can be adapted or combined in specialised data structures to target particular workloads. However, a standard tree normally follows one balancing scheme rather than applying the complete AVL and Red-Black invariants at the same time.

One of the biggest practical difficulties was memory and pointer management. Rotations and deletion modify several parent and child pointers at once. I experienced memory-related problems during development, and the current implementation still has some small memory leaks that I would like to fix. I am also sure that parts of the code can be further optimised.

Deletion was especially difficult because removing a black node can violate the black-height property. The algorithm then needs to consider the parent, sibling and sibling's children and apply the correct recolouring or rotation case.

(d) AI Use

I used ChatGPT throughout this project. I used it for explanations, debugging assistance, parts of the Red-Black Tree balancing and deletion logic, setting up the comparison AVL Tree, creating GoogleTest tests and CMake configuration, discussing the benchmark design, and helping to structure and generate this report.

I wrote and developed the Red-Black Tree incrementally, but I used ChatGPT more heavily when I became stuck on balancing and deletion. These methods contain several similar and mirrored cases, and I did not want to spend too much project time repeatedly debugging the same cases.

For the empirical study, ChatGPT also helped me create the AVL Tree comparison implementation and the initial GoogleTest/CMake testing setup. I reviewed and ran the tests myself. It also helped me design the benchmark so that both trees received the same input values and so that debug output was removed from the timed operations.

One example where the AI output needed to be adapted was deletion. Standard Red-Black Tree deletion is often explained using black NIL sentinel nodes, while my implementation uses nullptr. The standard approach therefore could not be copied directly. I needed to keep track of the parent separately and add explicit nullptr checks.

A second example was memory management and optimisation. AI helped me get the main algorithms working, but working output did not mean that the implementation was automatically memory-safe or optimal. I still have some small memory leaks, and I believe the code can be further optimised. This showed me that AI-generated code still needs testing and review rather than being accepted only because it compiles.

AI also suggested converting the Red-Black Tree to a generic C++ template. I experimented with this idea but decided not to keep it in the final implementation because it was not necessary for demonstrating the algorithm and added complexity that I did not need.

In terms of what I understood versus what I took on trust, I understand the main Red-Black Tree concepts used in the project: BST insertion and deletion, node colours, rotations, recolouring, red-parent violations and why balancing is required after removing a black node. I also understand the

purpose of the AVL comparison and how the benchmark measures insertion and search performance.

I relied more heavily on ChatGPT for some detailed pointer manipulation and deletion edge cases, and for scaffolding parts of the AVL and testing code. I tested and reviewed these parts, but I would not claim that I could reproduce every Red-Black deletion fix-up case from memory without referring to the algorithm.

Finally, ChatGPT was used to help generate and structure this written report and the performance plots. I provided the implementation details, benchmark results, project experience and points I wanted to discuss, while AI helped organise and present them.